

Spec v2: Chat médico ↔ paciente en tiempo real con videollamada Jitsi

Repos [api-medicos-por-venezuela](#) · [medicos-por-venezuela](#)

Estado borrador para revisión con Ori y Adarvelys

Fuente [tasks/mensajeria-medico-paciente/spec-chat-tiempo-real.md](#)

CONTENIDO

1. Objetivo
2. Lo que hay y se reutiliza
3. Decisiones de arquitectura
4. Modelo de datos
5. API REST
6. Cómo se asigna el médico (matching)
7. Protocolo WebSocket
3. Presencia en línea
9. Videollamada desde el chat
10. Interfaz
11. Seguridad, privacidad y riesgos
12. Configuración
13. Requisitos no funcionales
14. Dependencias de secuencia (solo las que la arquitectura impone)
15. Preguntas abiertas para el cliente

Fecha: 2026-10-02. Sustituye el alcance de `spec.md` (buzón asíncrono + WhatsApp) en lo que choque; lo que no choque (cifrado, grants, auditoría, correo de aviso) se conserva y se cita. Abarca dos repos: `api-medicos-por-venezuela` (dominio, esta spec es la canónica) y `medicos-por-venezuela` (UI). Reglas duras: `.claude/rules/mensajería.md` y `security.md`. Todo lo que aquí se afirma sobre «lo que hay» se leyó en el código el 2026-10-02.

1. Objetivo

Reemplazar la comunicación por WhatsApp entre médico voluntario y paciente por un **chat dentro de la plataforma**, al estilo del chat de Workana: lista de conversaciones a la izquierda, hilo a la derecha, entrega instantánea, aviso de mensaje nuevo, indicador de conexión del paciente y, desde el mismo hilo, la **videollamada Jitsi que ya existe**. El médico es el único que puede iniciar la videollamada; el paciente recibe la llamada entrante y la acepta o la rechaza.

Actores

Actor	Qué gana
Médico tratante	Un solo lugar para hablar con sus pacientes, ver si están conectados y llamarlos por video sin salir del hilo ni dar su número.
Paciente (con cuenta o anónimo con token)	Escribe a su médico desde la web, recibe respuesta al instante y atiende la videollamada desde el chat.
Administración	Ve que la conversación y las llamadas existen (conteos, estados, duración), nunca el contenido.

Fuera de alcance

- Puente con WhatsApp (API Cloud de Meta). Queda documentado en `spec.md` R9–R11 como fase posterior; esta spec no lo contradice pero no lo construye.
- Adjuntos, imágenes, audios, edición o borrado de mensajes.
- Push real (Web Push / FCM). `lib/firebase.ts` sigue sin conectar.
- Cita presencial como tipo de agenda (P6 de `.knowledge/mensajería.md`).
- Embeber Jitsi en un iframe. La CSP del frontend tiene `frame-src 'none'` (`next.config.js:42`); la sala sigue abriéndose en ventana aparte.

2. Lo que hay y se reutiliza

Pieza	Dónde	Cómo se usa aquí
Tabla <code>messages</code> cifrada, RLS deny-all, CHECK de ciphertext	<code>src/models/clinical.py:103-115</code> , migración <code>20260923_214425</code>	Se amplía (R1). Es el hilo.
Consulta como unidad clínica: <code>assigned_doctor_id</code> , <code>specialty_id</code> , <code>video_room_url</code> , <code>parent_consultation_id</code>	<code>src/models/consultation.py:43,51,71,117</code>	Un hilo = una consulta. La sala Jitsi ya vive ahí.
Cola por especialidad y claim atómico	<code>src/services/queue.py:70-84</code> (<code>take_consultation</code> , <code>FOR UPDATE NOWAIT</code>)	Es el <i>matching</i> . No se toca.
Sala Jitsi	<code>src/services/jitsi.py:8-10</code> (<code>vamed-{uuid}</code> en <code>JITSI_DOMAIN</code>), <code>get_or_create_room</code> <code>src/services/consultations.py:1004-1023</code> , <code>POST /consultations/{id}/video-room</code>	La llamada desde el chat reutiliza la misma sala de la consulta.
Token de consulta para paciente anónimo	<code>src/core/consultation_token.py</code> (HS256, <code>typ=consultation_access</code> , TTL 24 h), header <code>X-Consultation-Token</code>	Autentica al paciente sin cuenta en REST y en el WebSocket.
Principal, RBAC y pertenencia	<code>src/core/security.py:156-279</code> (<code>get_current_principal</code> , <code>require_permission</code>)	Permisos <code>messages.*</code> y <code>calls.start</code> para staff; el paciente nunca pasa por RBAC.
Grants y auditoría clínica	<code>src/services/clinical_access.py:117-170</code> (<code>audit_clinical_read</code> , <code>READ_CLINICAL_DATA</code>)	Toda lectura de cuerpos se concede por pertenencia y se audita.
Avisos por correo y preferencias	<code>src/services/notifications.py:35-56</code> (<code>NOTIFICATION_EVENTS</code> , <code>should_send</code>), <code>profiles.notification_prefs</code>	Evento nuevo <code>message_received</code> ; correo solo cuando el destinatario está desconectado.
SSE de sala de espera	<code>src/services/waiting_room.py:121-169</code> , <code>lib/waitingRoom.ts</code>	Se conserva para la fase de la sala. El chat usa WebSocket (D1).
Presencia por Supabase Realtime	<code>lib/presence.tsx</code> (<code>online-doctors</code>), <code>lib/patientPresence.tsx</code> (<code>room-{id}</code>)	Se conserva para lo que ya hace. La presencia del chat la calcula la API (D3).

Pieza	Dónde	Cómo se usa aquí
Modal previo a la videollamada	<code>components/AntesDeEntrarModal.tsx</code>	Se muestra al paciente al aceptar la llamada y al médico al iniciarla.
Notificaciones nativas del navegador	<code>lib/nativeNotifications.ts</code>	Mensaje nuevo y llamada entrante con la pestaña abierta.

Hechos de infraestructura que condicionan el diseño: la API corre en **un solo proceso** uvicorn (`Dockerfile:32`, sin `--workers`), **no hay Redis** y Supabase Realtime lo usa solo el frontend. El CI no ejecuta la suite de tests (solo `--collect-only`); la verificación es local.

3. Decisiones de arquitectura

D1. Tiempo real por WebSocket propio en la API, no por SSE ni por Supabase Realtime. Un canal `GET /api/v1/ws/chat` por pestaña. Motivos: (a) el paciente anónimo solo se autentica con el token de consulta, que únicamente entiende la API; (b) la presencia «en línea» debe salir de conexiones reales, no de lo que un cliente declare; (c) la llamada entrante necesita un empuje inmediato y un acuse (aceptar/rechazar) por el mismo canal; (d) el SSE actual es un *poll* de BD cada `WAITING_ROOM_POLL_SECONDS` (4 s), no instantáneo. Supabase Realtime Broadcast se descartó porque un canal público nombrado por UUID autoriza por oscuridad y un canal privado exige JWT de Supabase, que el paciente anónimo no tiene.

D2. El WebSocket transporta señales, nunca cuerpos. Igual que la regla del SSE: el evento `message.new` lleva `message_id`, `consultation_id`, `direction`, `sent_at` y un `preview` vacío. El cliente hace `GET /consultations/{id}/messages?after_id=` y recibe el cuerpo por REST, con grant y auditoría. Coste: una ida y vuelta extra (~50–150 ms en la misma región); beneficio: un único camino de lectura clínica, auditado, y ningún texto en memoria del hub.

D3. Presencia calculada en la API a partir de las conexiones. `ChatHub` mantiene en memoria `{principal_key → set(conexiones)}`. En línea = al menos una conexión viva con *pong* reciente. Al cerrar la última conexión se persiste `last_seen_at`. Para el paciente, `principal_key` es `patient:{consultation_id}` (sirve para anónimo y con cuenta); para el médico, `user:{user_id}`.

D4. Hub con interfaz, implementación en memoria, Redis cuando haya más de una réplica. `ChatHub` expone `publish(topic, event)`, `subscribe(topic)`, `presence(key)`. La implementación `MemoryHub` basta hoy (un proceso). Si la API escala a más de una tarea detrás del ALB, se activa `RedisHub` (`REALTIME_BACKEND=redis`, pub/sub + claves de presencia con TTL). Sin este cambio, dos réplicas mostrarían presencia y entregas inconsistentes: es el riesgo principal (sección 11).

D5. La videollamada es un recurso propio (`call_sessions`), no un estado de la consulta. Hoy la sala se abre desde `/entrar-videoconsulta` y la sala de espera sin registrar quién llamó, cuándo contestaron ni cuánto duró. Cada intento desde el chat crea una fila con su ciclo de vida (`ringing` → `accepted|declined|missed|cancelled` → `ended`) y deja un mensaje de sistema en el hilo. La sala Jitsi sigue siendo `consultations.video_room_url` (una por consulta, `get_or_create_room`).

D6. Asimetría de la llamada en tres capas. (1) UI: el componente del paciente no renderiza el botón. (2) API: `POST /consultations/{id}/calls` exige principal staff con permiso `calls.start` y ser el médico tratante; un paciente recibe 404. (3) Hub: el evento `call.incoming` solo se envía a `patient:{consultation_id}` . El paciente únicamente puede `accept` , `decline` y `end` .

D7. Hilo = consulta; el chat «se crea» al registrarse el paciente. No hay botón «nuevo chat». `POST /consultations` (registro) crea la consulta y, con ella, el hilo. El paciente puede escribir desde el primer segundo; hasta que un médico tome el caso, la UI dice «Tu mensaje quedará para el médico que tome tu caso». El *matching* es el claim por especialidad que ya existe (sección 6).

D8. Correo solo como respaldo. Si el destinatario está conectado al hub, no se envía correo. Si está desconectado, se envía uno por hilo cada `MESSAGING_MAIL_DEBOUNCE_MINUTES` (15) mientras siga sin leer. El correo nunca lleva el texto (`mensajeria.md` § Contenido).

D9. Jitsi no se toca. Ni autenticación de salas, ni dominio, ni *hash-config* (`lib/jitsi.ts`). Lo único nuevo alrededor es el registro de la llamada. Si el cliente quiere duración exacta medida por el servidor Jitsi (eventos de Prosody), es una iteración posterior.

4. Modelo de datos

4.1 `messages` (ampliada)

Columna	Tipo	Notas
<code>id</code>	uuid PK	existe
<code>consultation_id</code>	uuid FK consultations ON DELETE CASCADE	existe; es el hilo
<code>sender_role</code>	text CHECK <code>doctor</code> <code>patient</code> <code>system</code>	existe
<code>sender_user_id</code>	uuid FK users, nulo	nuevo; nulo para paciente anónimo y <code>system</code>
<code>direction</code>	text CHECK <code>doctor_to_patient</code> <code>patient_to_doctor</code> <code>system</code>	nuevo

Columna	Tipo	Notas
<code>kind</code>	text CHECK <code>text call</code> DEFAULT <code>text</code>	nuevo; <code>call</code> referencia una fila de <code>call_sessions</code>
<code>call_session_id</code>	uuid FK <code>call_sessions</code> , nulo	nuevo; obligatorio si <code>kind = call</code>
<code>body</code>	EncryptedText("messages.body")	existe; para <code>kind = call</code> va vacío cifrado o nulo (ajustar CHECK)
<code>client_msg_id</code>	text, nulo	nuevo; idempotencia al reenviar tras reconexión; único por (<code>consultation_id</code> , <code>client_msg_id</code>)
<code>sent_at</code>	timestampz DEFAULT now()	existe
<code>delivered_at</code>	timestampz, nulo	nuevo; primera vez que el destinatario recibió <code>message.new</code> por WS o lo listó por REST
<code>read_at</code>	timestampz, nulo	existe; lo marca el destinatario

Índices: (`consultation_id`, `sent_at`, `id`) ; único parcial (`consultation_id`, `client_msg_id`) WHERE `client_msg_id` IS NOT NULL ; (`consultation_id`) WHERE `read_at` IS NULL para contadores.

RLS sigue deny-all; todo acceso pasa por la API con `service_role` .

4.2 `call_sessions` (nueva)

Columna	Tipo	Notas
<code>id</code>	uuid PK	
<code>consultation_id</code>	uuid FK <code>consultations</code>	
<code>initiated_by_user_id</code>	uuid FK <code>users</code>	siempre el médico (D6)
<code>room_url</code>	text	copia de <code>consultations.video_room_url</code> en el momento de llamar
<code>status</code>	text CHECK <code>ringing accepted declined missed cancelled ended failed</code>	
<code>created_at</code>	timestampz	inicio del timbre
<code>ring_expires_at</code>	timestampz	<code>created_at</code> + <code>CALL_RING_TIMEOUT_SECONDS</code>

Columna	Tipo	Notas
<code>answered_at</code>	timestampz, nulo	cuando el paciente acepta
<code>ended_at</code>	timestampz, nulo	
<code>ended_by</code>	text CHECK <code>doctor patient system</code> , nulo	
<code>end_reason</code>	text, nulo	<code>hangup window_closed timeout error</code>
<code>duration_seconds</code>	int GENERATED (<code>ended_at - answered_at</code>)	nulo si no hubo <code>answered_at</code>
<code>patient_notified_via</code>	text CHECK <code>ws email none</code>	cómo se le avisó al paciente

Transiciones permitidas (escritura condicional `UPDATE ... WHERE status = :from`, nunca retroceder): `ringing → accepted | declined | missed | cancelled`; `accepted → ended | failed`. Índice (`consultation_id`, `created_at` desc).

4.3 Presencia

No hay tabla de presencia en vivo (D3). Se persisten solo marcas de «última vez»:

- `consultations.patient_last_seen_at` (timestampz, nulo): se actualiza al cerrar la última conexión del paciente y cada 60 s mientras esté conectado.
- `profiles.last_seen_at` (timestampz, nulo): ídem para el médico.

4.4 Permisos RBAC

Migración siembra `messages.read`, `messages.write`, `calls.start` para `doctor`, `specialist`, `admin`, `super_admin`. Tener `messages.read` no concede cuerpo: el grant de lectura es por pertenencia (`spec.md` R3, D2 del plan). El admin ve conteos, estados y duración de llamadas.

5. API REST

Prefijo `/api/v1`. Autenticación: `Authorization: Bearer <jwt Supabase>` para staff y pacientes con cuenta; `X-Consultation-Token` para paciente anónimo. Pertenencia validada en el servicio en **todas** las rutas, no en el router.

Método y ruta	Quién	Qué hace
GET /inbox? only_unread=&limit=&offset=	staff messages.read	Conversaciones donde el llamante es o fue médico tratante en la cadena. Por fila: consultation_id , code , nombre visible del paciente, especialidad, status de la consulta, last_message_at , last_direction , unread_count , patient_online , patient_last_seen_at , active_call (id o nulo). Sin cuerpos. Orden last_message_at desc .
GET /me/threads	paciente con cuenta	Lo mismo desde el lado del paciente: una fila por consulta propia (vigente por cadena), con doctor_online y unread_count .
GET /consultations/{id}/messages?after_id=&before_id=&limit=	médico tratante, paciente dueño	Página del hilo ordenada por sent_at, id . Cuerpo descifrado solo con grant; sin grant, null y la respuesta no falla. Una entrada READ_CLINICAL_DATA por página. Al listar como destinatario, marca delivered_at de lo pendiente.
POST /consultations/{id}/messages { body, client_msg_id }	médico tratante (messages.write), paciente dueño	Crea el mensaje, responde 201 con MessageResponse , publica message.new en el hub. body 1–4000 caracteres, sin HTML. Repetir client_msg_id devuelve 200 con el existente. 409 si la consulta no admite mensajes (R7 de la sección 7).
POST /consultations/{id}/messages/read	ambos	read_at = now() en mensajes de la dirección contraria sin leer; devuelve { marked } ; publica message.read al otro lado. Idempotente.
POST /consultations/{id}/typing	ambos	Señal efímera; no se persiste; se reenvía como typing al otro lado. Rate limit 1 por 2 s por conexión.
POST /consultations/{id}/calls	médico tratante con calls.start	Crea call_sessions en ringing , asegura video_room_url con get_or_create_room , publica call.incoming al paciente. Si el paciente no está conectado y hay correo, envía video_ready_email existente y marca patient_notified_via = email . 409 si ya hay una llamada ringing o accepted en ese hilo. Responde 201 con { call_id, room_url, ring_expires_at } .
POST /calls/{id}/accept	paciente dueño	ringing → accepted , answered_at . Responde { room_url } . Publica call.updated a ambos.
POST /calls/{id}/decline	paciente dueño	ringing → declined . Publica call.updated .

Método y ruta	Quién	Qué hace
POST /calls/{id}/cancel	médico iniciador	ringing → cancelled .
POST /calls/{id}/end { reason }	ambos	accepted → ended , ended_by , end_reason . Idempotente: si ya terminó, 200 con el estado actual.
GET /consultations/{id}/calls	médico tratante, paciente dueño, admin	Historial de llamadas de la consulta.

Toda ruta que muta el hilo registra `audit_log` (`message.sent` , `call.started` , `call.accepted` , `call.ended` ...) sin contenido. Un médico no tratante recibe 404, no 403, para no revelar existencia (criterio ya usado en `spec.md` CA2.1).

El paciente solo escribe mientras la consulta esté en `waiting` , `in_progress` , `contacted_whatsapp` , `scheduled` , `referred_to_specialist` o cerrada hace menos de `MESSAGING_AFTER_CLOSE_HOURS` (72, P5 pendiente). El médico escribe en los mismos estados salvo `waiting` (aún no es tratante). Rate limit del paciente: `PUBLIC_WRITE_RATE_LIMIT` por IP más 30 mensajes por hilo y hora.

6. Cómo se asigna el médico (matching)

No se diseña un *matching* nuevo; el chat se cuelga del que ya existe:

1. El paciente se registra en `/registro-paciente` y elige especialidad. La API crea la consulta en `waiting` con `specialty_id` y responde con el token de consulta; el frontend lo lleva a `/sala-espera?cid=...&t=...` (o a `/mi-caso` si inició sesión). Desde ese instante el hilo existe y puede escribir.
2. La consulta entra en la **cola de su especialidad** (`tasks/cola-por-especialidad/spec.md`). Los médicos ven en `/panel-medico` solo las consultas de las especialidades que tienen en `doctor_specialties` o como primaria en `profiles.specialty_id` .
3. Un médico la **toma** (POST `/consultations/{id}/claim` , `take_consultation` con `FOR UPDATE NOWAIT`): queda `assigned_doctor_id` , `status = in_progress` y se crea `video_room_url` . Dos médicos no pueden tomarla a la vez; el segundo recibe 409.
4. El hub publica `thread.assigned` al paciente con el nombre visible del médico; el hilo cambia su cabecera de «Esperando médico de Cardiología» a «Dra. X · Cardiología». Los mensajes que el paciente escribió antes ya están en el hilo y el médico los ve con su `unread_count` .

5. Si el médico **deriva** a otra especialidad, nace una consulta hija (`parent_consultation_id`). El hilo vigente del paciente es el de la hija (`current_in_chain`); el médico anterior conserva lectura del tramo que atendió y nada del nuevo. En la UI del paciente la lista muestra una sola conversación por cadena, con un separador «Derivado a Dermatología» en el hilo.

Un paciente con cuenta y varias consultas ve varias conversaciones (una por cadena). Un paciente anónimo ve una sola, la de su token.

7. Protocolo WebSocket

Endpoint `GET /api/v1/ws/chat` . Subprotocolo `mpv-chat.v1` . El navegador no puede mandar cabeceras en el *handshake*, y el token en la URL quedaría en logs del ALB, así que la autenticación va en el **primer frame**:

```
{ "type": "auth", "bearer": "<jwt>" } // staff o paciente con cuenta
{ "type": "auth", "consultation_token": "<jwt>" } // paciente anónimo
```

Si no llega un `auth` válido en 5 s, el servidor cierra con código 4401. Un principal se suscribe automáticamente a los *topics* a los que tiene pertenencia: el médico a todas sus consultas con hilo y a `user:{id}` ; el paciente a `patient:{consultation_id}` de su cadena vigente.

Eventos servidor → cliente (todos con `ts`):

type	Payload	Cuándo
<code>ready</code>	<code>{ principal_key, subscriptions: [consultation_id] }</code>	tras <code>auth</code>
<code>message.new</code>	<code>{ consultation_id, message_id, direction, sent_at }</code>	mensaje creado; el cliente hace <code>GET ...?after_id=</code>
<code>message.read</code>	<code>{ consultation_id, up_to_message_id, read_at }</code>	el otro lado marcó leído
<code>typing</code>	<code>{ consultation_id, who: doctor patient }</code>	efímero, expira en 3 s en el cliente
<code>presence</code>	<code>{ consultation_id, who, online: bool, last_seen_at }</code>	el otro lado conectó o desconectó
<code>thread.assigned</code>	<code>{ consultation_id, doctor_display_name, specialty }</code>	claim o derivación
<code>call.incoming</code>	<code>{ call_id, consultation_id, room_url, ring_expires_at }</code>	solo al paciente

type	Payload	Cuándo
call.update d	{ call_id, status, answered_at?, ended_at? }	cualquier transición
inbox.badge	{ unread_total }	para el contador global del médico
ping	{}	cada 25 s (por debajo del <i>idle timeout</i> de 60 s del ALB)

Eventos cliente → servidor: solo `auth` y `pong`. Escribir, marcar leído, *typing* y las llamadas van por REST para no duplicar validación ni auditoría.

Reconexión: *backoff* exponencial 1, 2, 4, 8, máx. 30 s con *jitter*. Al reconectar, el cliente pide `GET .../messages?after_id=<último visto>` por cada hilo abierto y `GET /inbox`. Si el WebSocket falla tres veces seguidas (proxy corporativo, red móvil hostil), el cliente pasa a *polling* REST cada 10 s y lo indica con «Conexión limitada» en la cabecera; la presencia propia en ese modo se declara con `POST /presence/heartbeat` cada 30 s.

8. Presencia en línea

- **Definición:** en línea = al menos una conexión WS autenticada con `pong` en los últimos 40 s, o un *heartbeat* REST en los últimos 45 s (modo degradado).
- **Pestaña en segundo plano:** sigue en línea mientras el socket viva; los navegadores móviles lo cortan al poco tiempo y el estado pasa a desconectado solo, sin lógica extra.
- **Qué ve el médico:** punto verde «En línea» o gris «Desconectado · últ. vez hace 12 min» / «Aún no ha entrado» (sin `last_seen_at`). En la lista de conversaciones, el punto junto al nombre.
- **Qué ve el paciente:** «Tu médico está en línea» / «Tu médico no está conectado; te avisaremos por correo cuando responda». Es una decisión revisable (D-UI-1): cuesta lo mismo y evita que el paciente espere una respuesta inmediata cuando no la habrá.
- La presencia **no** se usa para autorizar nada; solo informa y decide si se manda correo (D8) y cómo se avisa la llamada (`patient_notified_via`).
- Convive con `online-doctors` y `room-{id}` de Supabase Realtime, que siguen sirviendo al panel y a la sala de espera. No se migran en esta iteración.

9. Videollamada desde el chat

Secuencia feliz:

1. El médico pulsa **«Iniciar videollamada»** en la cabecera del hilo. En el mismo gesto de clic el frontend llama `POST /consultations/{id}/calls` y, con la respuesta, abre `browserRoomUrl(room_url)` con `window.open` (debe ocurrir dentro del gesto para que el navegador no bloquee la ventana: el `open` se hace síncrono con una URL `about:blank` y se asigna `location` al llegar la respuesta). Se muestra `AntesDeEntrarModal` solo la primera vez por sesión.
2. El paciente conectado recibe `call.incoming`: aparece un **banner fijo** en la parte superior del chat con foto/nombre del médico, «Videollamada entrante», botones **Aceptar** y **Rechazar**, sonido corto (si el usuario interactuó antes con la página) y notificación nativa si la pestaña no está enfocada. Cuenta atrás visible hasta `ring_expires_at`.
3. **Aceptar**: `POST /calls/{id}/accept`, y en el mismo clic `window.open(room_url)`. El hilo muestra «Videollamada en curso · 00:42» en ambos lados, con botón **Finalizar**.
4. **Finalizar** en cualquier lado, o detección de ventana cerrada (`popup.closed` cada 2 s, ambos lados), llama `POST /calls/{id}/end`. El hilo deja un mensaje de sistema «Videollamada · 14 min».

Caminos alternativos:

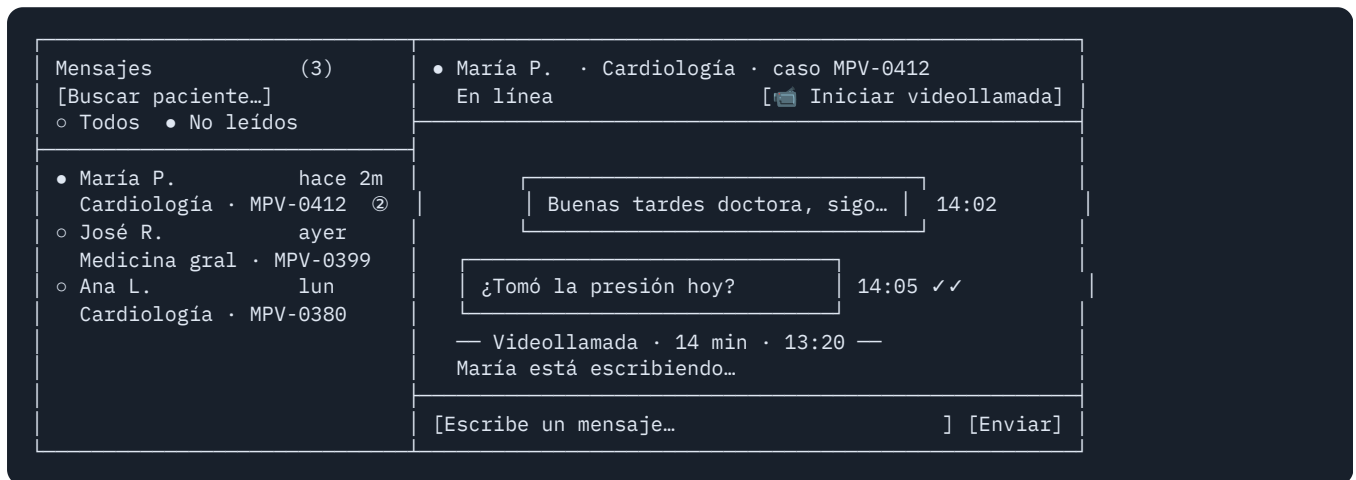
- **Rechazar**: `declined`; el médico ve «El paciente rechazó la llamada» en el hilo y puede escribir.
- **Sin respuesta**: una tarea de fondo en la API (`asyncio`, cada 10 s) pasa a `missed` las `ringing` vencidas y publica `call.updated`. El paciente, al volver, ve «Llamada perdida · 12:03» y un botón **«Avisar que estoy disponible»** que envía un mensaje de texto predefinido (no llama: la asimetría se mantiene).
- **Paciente desconectado al llamar**: el botón del médico cambia a **«Llamar y avisar por correo»** con tooltip «El paciente no está conectado». La llamada igual timbra `CALL_RING_TIMEOUT_SECONDS` (120 s en este caso, 45 s si está en línea) y se envía el correo «Tu médico te espera» con enlace a `/entrar-videoconsulta` o a `/mi-caso`, que ya existe.
- **Cancelar**: el médico cierra el timbre antes de respuesta.
- **Dos pestañas del paciente**: `call.incoming` llega a todas; la primera que acepta gana; las demás reciben `call.updated` y ocultan el banner.
- **Llamada ya activa**: 409 en `POST /calls`; la UI deshabilita el botón mientras `active_call`.

Lo que **no** cambia: la ruta `/entrar-videoconsulta`, el botón «Entrar» de la sala de espera, el «Contactar por WhatsApp» del modal previo y la ausencia de autenticación en la sala Jitsi.

10. Interfaz

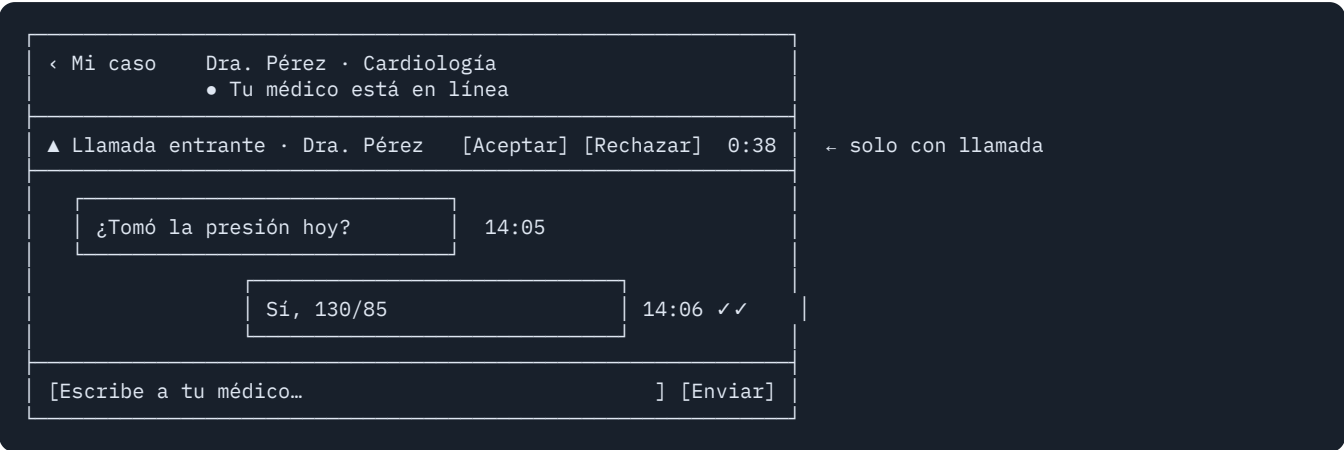
Sin Tailwind ni librería de componentes: CSS global con las variables de `styles/globals.css` (`--brand #0066fe` , `--navy` , `--green` , `--red` , `--muted`), fuente Nunito Sans, clases `btn` , `card` , `notice` . Mobile-first: el médico usa el buzón desde el teléfono.

10.1 Vista del médico — `/panel-medico/mensajes` (y `/panel-medico/mensajes/[id]`)



- Lista: punto de presencia, nombre visible (el mismo que ya usa el panel), especialidad y código, hora relativa del último mensaje, burbuja de no leídos, filtro «No leídos».
- Cabecera del hilo: presencia con texto, enlace «Ver caso» a `/panel-medico/consulta/[id]` , y el botón **Iniciar videollamada** (variante «Llamar y avisar por correo» si el paciente no está; deshabilitado si la consulta está cerrada fuera de ventana o hay llamada activa).
- Burbujas: propias a la derecha con marcas ✓ enviado, ✓✓ entregado, ✓✓ azul leído; ajenas a la izquierda. Separadores de día. Mensajes de sistema centrados (llamadas, derivación, cierre).
- Composer: `textarea` que crece, `Enter` envía, `Shift+Enter` salto; deshabilitado con aviso si el hilo no admite mensajes.
- En móvil (< 768 px) la lista y el hilo son dos pantallas; atrás vuelve a la lista.
- En `/panel-medico` (cabecera global) se añade el icono «Mensajes» con `unread_total` (`inbox.badge`). En `/panel-medico/consulta/[id]` se añade una pestaña «Mensajes» que renderiza el mismo componente de hilo.

10.2 Vista del paciente — `/mi-caso` (con cuenta) y `/sala-espera` (token)



Diferencias deliberadas respecto al médico:

Elemento	Médico	Paciente
Botón iniciar videollamada	Sí	No existe (ni oculto por CSS: el componente no lo renderiza)
Llamada entrante	No aplica	Banner con Aceptar / Rechazar, sonido y notificación nativa
Lista de conversaciones	Todas sus consultas	Solo las tuyas; una sola si es anónimo
Presencia del otro	«En línea / últ. vez hace...»	«Tu médico está en línea / no está conectado»
Marcas de lectura	✓ ✓ ✓ ✓ azul	Solo «Leído» bajo su último mensaje leído
Antes de tener médico	—	Cabecera «Esperando médico de Cardiología», composer activo con aviso
Acceso	Sesión + RBAC + pertenencia	Sesión y <code>owns_patient</code> , o <code>X-Consultation-Token</code>

En `/sala-espera` el hilo va debajo de `SalaEsperaEnVivo`, que sigue mostrando la fase. En `/mi-caso` el hilo es una tarjeta más junto a «Mis consultas».

10.3 Accesibilidad

Banner de llamada con `role="alertdialog"` y foco en «Aceptar»; lista de mensajes con `aria-live="polite"`; presencia con texto, no solo color; todo operable con teclado; contraste AA con los colores de marca.

11. Seguridad, privacidad y riesgos

- Cuerpos cifrados en reposo (`EncryptedText`) y legibles solo por grant de pertenencia; toda lectura se audita. Nada del cuerpo en WS, correo, logs ni Excel.
- Teléfonos y correos del otro lado nunca aparecen en esquemas de mensajería ni de llamadas.
- Token de consulta por primer frame del WS, no por URL. Token caducado durante una sesión larga: el servidor cierra con 4401 y el frontend pide uno nuevo con el flujo ya existente de la sala.
- `POST /calls` triple barrera (D6). Test negativo obligatorio: paciente con token válido recibe 404 al intentar llamar.
- Idempotencia: `client_msg_id` en mensajes; transiciones condicionales en llamadas; tests de concurrencia para doble `accept` y doble `read` .
- **Riesgo 1 — escalado horizontal.** `MemoryHub` solo es correcto con una réplica. Antes de subir `desired_count` del servicio hay que activar `RedisHub` . Se documenta en el README de despliegue y la API loguea un WARNING al arrancar si `REALTIME_BACKEND=memory` en producción.
- **Riesgo 2 — ALB y WebSocket.** El ALB soporta WS, pero el *idle timeout* por defecto es 60 s: `ping` cada 25 s. Confirmar que el *listener* no termina en un *target* HTTP/1.0.
- **Riesgo 3 — CSP.** `connect-src` ya incluye `API_URL` ; hay que añadir el esquema `wss:` del mismo host o el navegador bloqueará el socket en silencio.
- **Riesgo 4 — sala Jitsi sin autenticación.** Quien tenga la URL entra. Es el estado actual y el cliente pidió no tocar Jitsi; la llamada desde el chat no lo empeora porque la URL solo viaja a los dos participantes autenticados. Queda anotado para una iteración futura (JWT de Jitsi).
- **Riesgo 5 — otro desarrollador activo.** Leonardo Alvarado hace todos los merges; avisar qué archivos toca cada entrega (`clinical.py` , `consultations.py` , `notifications.py` , `main.py`).

12. Configuración

`REALTIME_BACKEND` (`memory` | `redis` , por defecto `memory`), `REDIS_URL` (solo si `redis`), `WS_PING_SECONDS` (25), `WS_AUTH_TIMEOUT_SECONDS` (5), `PRESENCE_TTL_SECONDS` (40), `CALL_RING_TIMEOUT_SECONDS` (45), `CALL_RING_TIMEOUT_OFFLINE_SECONDS` (120), `MESSAGING_AFTER_CLOSE_HOURS` (72), `MESSAGING_MAIL_DEBOUNCE_MINUTES` (15), `MESSAGING_PATIENT_HOURLY_LIMIT` (30), `MESSAGING_MAX_BODY_CHARS` (4000). Todas en `.env.example` con comentario. Frontend: `NEXT_PUBLIC_API_WS_URL` (derivable de `NEXT_PUBLIC_API_URL` cambiando el esquema; se expone por si el WS va por otro host).

13. Requisitos no funcionales

- Latencia de señal (`POST` → `message.new` en el otro cliente) < 500 ms en la misma región; cuerpo visible < 1 s.
- Reconexión transparente sin pérdida ni duplicados (`after_id` + `client_msg_id`).
- 200 conexiones WS simultáneas por proceso sin degradar la API REST (prueba de carga local con `websockets` antes de desplegar).
- Cobertura ≥ 95 % en módulos nuevos; tests de concurrencia en claim-con-mensajes-previos, doble `accept` , doble `read` , mensaje repetido por `client_msg_id` .
- E2E Playwright: médico escribe y paciente anónimo lo ve sin recargar; médico llama, paciente acepta y se abre la ventana (interceptar `window.open`); paciente no ve el botón de llamada.

14. Dependencias de secuencia (solo las que la arquitectura impone)

1. Migración (`messages` ampliada, `call_sessions` , permisos, `last_seen_at`) antes que cualquier servicio.
2. `services/messaging.py` + REST de mensajes antes que el hub: el hub solo publica lo que REST ya persiste, y la UI puede funcionar en modo *polling* sin WS.
3. `ChatHub` + `/ws/chat` + presencia antes que llamadas: `call.incoming` y `patient_notified_via` dependen de saber si el paciente está conectado.
4. Llamadas (`call_sessions` , REST, tarea de vencimiento) antes que su UI.
5. UI del médico antes que la del paciente: el paciente anónimo se prueba contra un hilo que ya tiene mensajes del médico.
6. CSP (`wss:`) y verificación del ALB en el primer despliegue a `dev_aws` , antes de activar el WS para usuarios reales.

15. Preguntas abiertas para el cliente

- **Q1.** ¿El paciente debe ver si su médico está en línea (D-UI-1) o solo el médico ve al paciente?
- **Q2.** Ventana para escribir tras cerrar la consulta: 72 h propuestas (P5 heredada).
- **Q3.** Llamada con paciente desconectado: ¿timbrar 120 s y avisar por correo, o solo avisar por correo sin timbrar?

- **Q4.** ¿Se conserva «Contactar por WhatsApp» en el modal previo a la videollamada ahora que existe el chat? Ori pidió proteger el número del médico.
- **Q5.** ¿Hay previsión de más de una réplica de la API en AWS? Decide si Redis entra ya o después.
- **Q6.** ¿La fase WhatsApp (`spec.md` R9–R11) sigue en el encargo o se pospone indefinidamente?

Documento vivo. La copia canónica está en el repo de la API; los cambios acordados en la revisión se vuelcan allí.